DataSherpa AI — Technical Guide

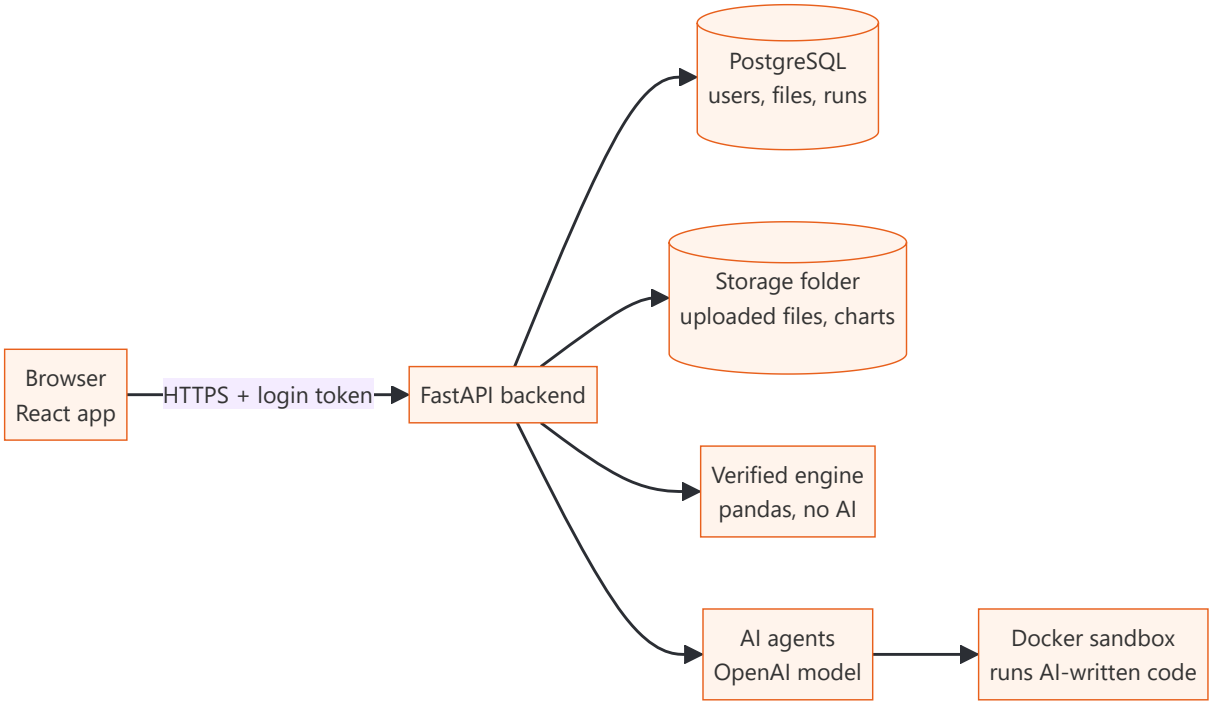
Navigate data. Discover answers. Make better decisions.

> A plain-English guide to how DataSherpa works, written so you can understand the system later **without reading the code**. Each topic says *what it does*, *why it's there*, and *where it lives*. Setup steps are in README.md; test evidence is in VALIDATION.md; agreed UI changes are in CHANGE_NOTES.md.

1. What DataSherpa does

A user uploads a sales file (CSV or Excel), asks a business question in plain English ("Why did sales drop last month?"), and gets a **formal report**. Every number in that report is calculated from the file, **checked a second way**, and **approved by a person** before it counts as final.

The main idea: **the AI never does arithmetic.** Numbers come from real calculations. The AI plans, writes code, explains and suggests next steps.



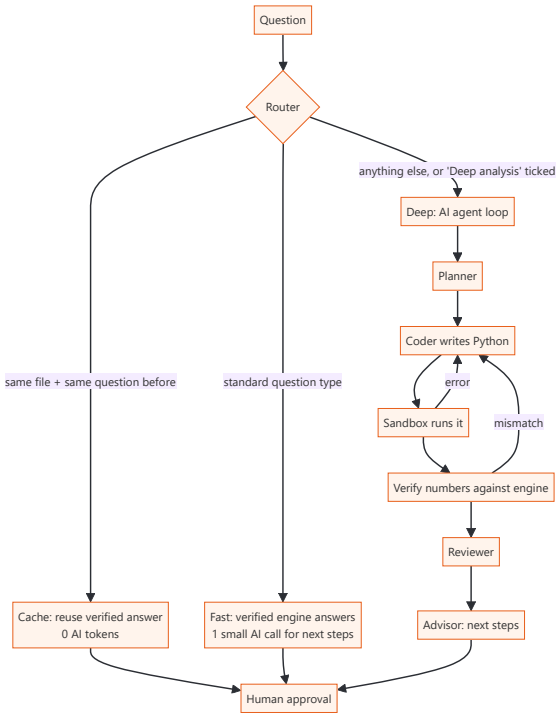
Part	Technology
Website	React + TypeScript + Chakra UI (`frontend/`)
Server	FastAPI, Python 3.11 (`backend/app/`)
Database	PostgreSQL + SQLAlchemy + Alembic migrations
AI	Microsoft Agent Framework + OpenAI model (set in `.env`)

Part	Technology
Safe code runner	Docker image `datasherpa-sandbox` (`sandbox-image/`)

2. Journey of a question

Upload (once per file). The file is read, its columns are recognised (date, product, region, price…) and the **Data Quality Guardian** checks every row (see section 6). Problem rows are recorded; the original file is never changed.

Asking a question. A *router* picks the cheapest safe path:



- **Cache** – same file (by fingerprint) and same question → the earlier verified answer is reused.
- **Fast** – standard types (sales change, profit, data quality) are answered by the engine directly. About 1k tokens and ~16 s, versus ~24k tokens and ~2 min for deep.
- **Deep** – the AI loop: **Planner** (what to calculate) → **Coder** (writes `analysis.py`) → **Sandbox** (runs it safely) → **Verification** (re-computes key numbers independently) → **Reviewer** (checks the result makes sense) → **Advisor** (ranked next steps).

Loop limits (the application enforces these, not the AI):

Limit	Value	When reached
AI messages per run	12	Run stops: "message limit"
Code attempts	3	Run stops: "attempt limit"
Time per code run / whole run	30 s / 10 min	Run stops: "time limit"
Time per AI call	4 min	Treated as an error
Tokens per run	80,000	Run stops: "token limit"

A stopped run can show partial findings, but they are clearly marked **"Incomplete — not approved"**.
Where: `backend/app/agents/` (router, orchestrator, fast path), `services/run_manager.py` (background runs, cache).

3. Features covered

Area	What's built
Accounts	Register, sign in, sign out, profile name, **forgot password** (one-time reset link)
Files	Upload CSV/XLSX (max 10 MB, 100k rows), choose Excel sheet, fix column meanings, archive/restore/delete, **10 active files**, older ones auto-archived
Data quality	Duplicates, invalid dates/numbers, negatives, missing values, spelling variants, revenue-rule mismatches, outliers
Questions	Plain-English questions, follow-ups in the same session, optional "Deep analysis"
Report	Header with report type/number/date, 6 sections (overview, ask, findings, actions, quality & validation, downloads & approval)
Downloads	Save as PDF (print view) and an **Excel report** styled like the screen (charts, analysed rows, all checks)
Approval	Approve or request changes (a corrected run starts automatically)
Look	Orange/charcoal theme on all pages, glowing buttons, Poppins headings

4. Guardrails and security

Guardrails – keep answers honest

Rule	How it's enforced
No AI arithmetic	Numbers come only from executed code or the engine
Numbers checked twice	Plain-Python recomputation (`math.fsum`) must match the analysis; a mismatch blocks approval
Facts ≠ causes	Findings are labelled *Fact* or *Possible reason (not proven)*
Don't invent insights	No profit without cost columns; costs are never guessed from prices
Reproducible	Re-running the code on the same file must give the same numbers
AI ≠ human approval	The Reviewer's verdict and the person's decision are stored separately; only a click approves
File content is data	Cell text is never treated as instructions to the AI; only compact summaries are sent
Source file untouched	Analysis uses a working copy; every exclusion is recorded

Security – keep users and the server safe

Area	What protects it
AI-written code	Runs only in Docker: no network, read-only input, non-root user, 512 MB RAM, 1 CPU, process/file/output limits, removed afterwards
Passwords	bcrypt hashes; minimum 10 characters
Login tokens	30-minute access token in memory; 7-day refresh token in an httpOnly, SameSite=Strict cookie, rotated on use; reuse of an old token signs out every session
Password reset	Random one-time link, only its hash stored, 30-minute expiry, newest link wins, signs out everywhere; same reply whether the email exists or not
Abuse	10 requests/minute per IP on sign-in endpoints
Data access	Every file, run and download is checked against its owner
Excel downloads	Cells that look like formulas are escaped
Secrets	API keys only in `.env` (never in code or logs)

Where: `app/auth/`, `app/sandbox/`, `app/verification/`, `app/services/auth_service.py`.

5. Error handling

All server errors return one shape — `{"error": {"code", "message"}}` — and the website shows the message in plain words.

What goes wrong	What the user sees	What the system does
Wrong file type / too big / too many rows	Clear upload error	File rejected before it's stored
Data too broken (e.g. no dates)	"Analysis blocked" with the reason	Questions are refused; no AI tokens used
Question needs missing data (e.g. profit without cost)	"Can't be answered safely with this data"	Stopped by validation
Question is unclear	"One question before analysing"	Planner asks a clarifying question
AI code crashes	Progress shows "Correcting"	Error is fed back to the Coder; up to 3 attempts
Numbers don't match	"Not verified", no Approve button	Result blocked
Limits reached / timeout / cancel	Stop reason shown, partial results marked not approved	Run ends cleanly
AI service down or not configured	"Failed" with a plain message	Nothing fabricated
Server restarted mid-run	Run shown as interrupted	Marked on start-up
Session expired	Silent refresh, or back to sign-in	Refresh token rotation

What goes wrong	What the user sees	What the system does
Reset link old/used	"Invalid or expired — ask for a new one"	Link refused
Database off (Docker not running)	"Request failed (500)"	Start Docker Desktop, then the database

6. Data quality, reports and downloads

Data Quality Guardian (`app/profiling/`) checks every row. Each problem gets a severity (note, warning, critical) and one handling: *left out* (duplicates, invalid dates), *treated as unknown* (bad numbers), *kept and flagged* (outliers, rule mismatches), *spelling merged*, or *blocked*. Flagged rows are named in the report ("please check rows 32, 48"); if leaving them out would flip the answer, the report says so.

Report (`frontend/src/components/analysis/RunResult.tsx` , `lib/report.ts`): header + question banner, then 1 Business Overview (KPI cards, rings) · 2 Ask Your Business Data · 3 AI Findings (facts vs. possible reasons, bar charts) · 4 Recommended Actions (priority) · 5 Data Quality & Validation (rows to check, "How the numbers were worked out") · 6 Reports & Next Steps (downloads, approval). Calculation code is viewable only under *More details*.

Excel (`backend/app/reports/export.py` , `GET /api/v1/runs/{id}/export`): sheets *Report* (same layout with Excel charts), *Analysed data* (flagged rows highlighted), *All checks*. **PDF** is the browser's "Save as PDF"; menus and buttons are hidden when printing.

7. Testing and proof

- **Backend:** 240 tests (plus sandbox and live-AI tests run separately) — auth, uploads, quality scenarios, engine maths, verification, routing, agent stop rules, exports, password reset.
- **Frontend:** 22 tests — report, approval, verification table, forms, password pages.
- **Evaluation with the real AI:** 24/24 known answers correct, 84/84 checks passed, 6/6 reproducible (details in VALIDATION.md).
- **Checks before every commit:** `ruff check` + `pytest` (backend), `lint` + `type-check` + `vitest` (frontend).

8. Code map, limits and future work

Folder	Job
`backend/app/ingestion/`	Read CSV/XLSX safely, pick the Excel sheet
`backend/app/profiling/`	Recognise columns, Data Quality Guardian
`backend/app/analysis/`	Verified engine: month comparison, drivers, profit
`backend/app/verification/`	Independent recomputation, reproducibility
`backend/app/agents/`	Router, fast path, AI agents, orchestrator and stop rules

Folder	Job
`backend/app/sandbox/`	Docker runner for AI-written code
`backend/app/reports/`	Excel report export
`backend/app/services/`	Glue: datasets, runs, auth, email
`backend/app/routers/`	API endpoints (`/api/v1/...`)
`backend/alembic/`	Database migrations
`frontend/src/pages/`	Screens: sign-in, datasets, upload, dataset, ask/report, reset password
`frontend/src/lib/report.ts`	Turns verified results into KPI cards, charts and plain names
`sandbox-image/`	Dockerfile for the safe runner

Settings live in `.env` (see `.env.example`): database, secret key, OpenAI model and key, limits, and optional SMTP for reset emails.

Known limits / future work

- **Email:** no email service yet. Reset links appear in the server log in development; add SMTP settings to send real emails.
- **Change password** on the Profile page is not built yet.
- **Rate limiting** is per server process; use a shared store (e.g. Redis) when scaling out.
- **Landing page** and in-sync page designs are approved as previews only (see CHANGE_NOTES #11, #12).
- **Deployment** (hosting, HTTPS, backups) is not set up yet.
- **Business types:** only the retail-sales profile exists; others need a new profile file.